Nix and Crane artifact reuse

> **Status:** Historical Rust CI investigation and current documentation cache record. The experiment retains its PR 108 context; the documentation source and consumer contracts were reviewed 2026-10-13.
>
> The Rust CI experiments below are historical. Uses of "now" or "current" describe the surrounding experiment, not the final implementation. See CI maintenance and measurements for the maintained CI contract, retained results, branch migration and reporting workflow. The documentation producer and live-watcher sections retain their separate build and cache contract. The earlier source ownership split, crate-level checks, cross-run producer, cross-job barriers and caveats remain as the investigation history behind those designs; the maintained CI guide now separates the final Rust CI contract from this experiment log.

SOURCE31 observes the source lists at `a517ca563d78d161cff17fe60e661f80529c9a83`: embedded native rendering resources belong to `linnet-render-py`; compile-time test roots and production resources remain distinct inputs of their owning packages. This observation is not a producer-closure success. SOURCE30 separately reconciles missing compile inputs and must be integrated before final documentation qualification. Historical cache timings below remain tied to their original experiment.

This note records the cache-reuse audit for the NixCI Rust build graph.

Sources checked

- Crane FAQ: https://crane.dev/faq/constant-rebuilds.html
- Crane quick-start example: https://crane.dev/examples/quick-start.html
- Crane API reference: https://crane.dev/API.html
- Crane issue 180 and comments: https://github.com/ipetkov/crane/issues/180

The important Crane contracts are:

- Nix rebuilds a derivation when any direct input changes, including files in `src`. Source filtering and narrow source sets prevent unrelated invalidation.
- `cargoArtifacts` contains an existing Cargo `target` tree, as a directory or archive. Crane restores it after patching and before build hooks, so Cargo can mark already-built units as fresh.
- The inherited artifact only avoids compilation when Cargo asks for the same unit: same profile, target kind, package graph, and feature set.
- `doNotLinkInheritedArtifacts = true` does not disable reuse. It asks Crane to deep-copy inherited artifacts instead of symlinking reusable `.rlib`/`.rmeta` files. That can be slower, but it is not the reason Cargo recompiles.
- Crane issue 180 has no built-in `buildWorkspace` answer. The maintainer points at a Guppy/topological chain of package derivations with filtered sources and `cargoArtifacts` passed from dependencies. Later comments describe the same pattern: per-crate dependency artifacts trade more derivations for better code-change reuse.

Tested MWEs

The MWE workspace was created under `/tmp/crane-reuse-mwe` with crates `a` and `b`, where `b` depends on `a`. Commands used the repository's pinned Rust:

```
PATH=/nix/store/qqq8y0xm1n3y9wy8a7hb05clyr9dbcb9-rust-stable-with-components-2026-04-16/bin:$PATH
```

The source files were:

```
# Cargo.toml
[workspace]
members = ["a", "b"]
resolver = "2"

[profile.ci]
inherits = "dev"
debug = false
```

```
# a/Cargo.toml
[package]
name = "a"
version = "0.1.0"
edition = "2021"

[features]
extra = []

[lib]
path = "src/lib.rs"
```

```
// a/src/lib.rs
pub fn value() -> u32 {
    1
}
```

```
# b/Cargo.toml
[package]
name = "b"
version = "0.1.0"
edition = "2021"

[dependencies]
a = { path = "../a" }

[lib]
path = "src/lib.rs"
```

```
// b/src/lib.rs
pub fn value() -> u32 {
    a::value() + 1
}
```

Source path alone was not enough to force rebuilds

Build `a` in one copy of the workspace, copy `target`, then build `b` in an identical workspace at another path:

```
CARGO_INCREMENTAL=0 cargo build \
  --manifest-path /tmp/crane-reuse-mwe/source-a/Cargo.toml \
  --target-dir /tmp/crane-reuse-mwe/target-a \
  --profile ci -p a -vv

cp -a /tmp/crane-reuse-mwe/target-a /tmp/crane-reuse-mwe/target-b

CARGO_INCREMENTAL=0 CARGO_LOG=cargo::core::compiler::fingerprint=info cargo build \
  --manifest-path /tmp/crane-reuse-mwe/source-b/Cargo.toml \
```

```
  --target-dir /tmp/crane-reuse-mwe/target-b \
  --profile ci -p b -vv
```

Observed result: Cargo printed `Fresh a` and only compiled `b`. The absolute workspace path alone did not explain the reuse failure.

Feature mismatch does force a second crate variant

Add feature `extra` to crate `a`, build `a` with that feature, then consume the copied target from `b` without the feature:

```
CARGO_INCREMENTAL=0 cargo build \
  --manifest-path /tmp/crane-reuse-mwe/source-a/Cargo.toml \
  --target-dir /tmp/crane-reuse-mwe/target-feature-a \
  --profile ci -p a --features a/extra -vv

cp -a /tmp/crane-reuse-mwe/target-feature-a /tmp/crane-reuse-mwe/target-feature-b-
narrow

CARGO_INCREMENTAL=0 CARGO_LOG=cargo::core::compiler::fingerprint=info cargo build \
  --manifest-path /tmp/crane-reuse-mwe/source-c/Cargo.toml \
  --target-dir /tmp/crane-reuse-mwe/target-feature-b-narrow \
  --profile ci -p b -vv
```

Observed result: Cargo compiled `a` again. The log looked for a different fingerprint (`a-0a0f...`) than the already-built feature-enabled artifact (`a-5b84...`).

Running the same consumer with matching qualified features:

```
cp -a /tmp/crane-reuse-mwe/target-feature-a /tmp/crane-reuse-mwe/target-feature-b-
wide

CARGO_INCREMENTAL=0 CARGO_LOG=cargo::core::compiler::fingerprint=info cargo build \
  --manifest-path /tmp/crane-reuse-mwe/source-c/Cargo.toml \
  --target-dir /tmp/crane-reuse-mwe/target-feature-b-wide \
  --profile ci -p b --features a/extra -vv
```

Observed result: Cargo printed `Fresh a` and only compiled `b`.

Local audit findings

The repo uses Hakari and Guppy to create a cacheable workspace graph, but Cargo artifact reuse depends on more than the Nix graph shape. Cargo reuses an inherited artifact only if the requested unit has the same package, profile, target kind, source freshness metadata, feature set, and dependency hashes.

Artifact merge and freshness fixes

The artifact merge layer had to preserve Crane's previous-artifact chain and keep inherited targets writable:

```
if [ -e "$artifact.prev" ] || [ -L "$artifact.prev" ]; then
  unpack_artifact "$(realpath "$artifact.prev")"
fi
zstd -d "$artifact" --stdout | tar --no-same-permissions -x -C target
rsync -a --chmod=u+w "$artifact/" target/
```

The merge layer materializes even a single inherited artifact before publishing one full compressed archive. This preserves directory-valued `target.tar.zst.prev` links as well as the file-valued links Crane follows. Returning an artifact with an unhandled directory-valued base previously made later

derivations lose the root prebuild and rebuild the heavy dependency stack. The expanded writable tree now stays in the build directory; the output contains only `target.tar.zst`, without a previous-artifact link or expanded target tree. CLI packaging restores that archive and declares its runtime library inputs explicitly: compression hides embedded store paths, so those inputs must be available to Nix when it discovers references in the extracted binaries.

Dependency-only archives also strip dummy workspace artifacts after compiling third-party dependencies. The strip logic intentionally ignores binary target names: `clinnet` has a binary named `linnet`, and stripping by every target name would delete `liblinnet-*`. Stripped producers ask Crane to install a full archive of that target directly, so deleted dummy artifacts cannot return through a previous-artifact link. This avoids installing a delta and then unpacking, stripping, and recompressing its entire inheritance chain during fixup.

The Hakari package needs a real build artifact, not just a dependency-only archive. Otherwise downstream Cargo fingerprints point at a different `gammaloop-workspace-hack` unit. The flake now builds:

- `workspaceHackDependencyArtifacts`: third-party seed for the hack.
- `workspaceHackBuildArtifacts`: real `cargoBuild` of the hack from that seed.

The hack build script also needed stable source freshness metadata. Cargo records one local-file marker for build-script freshness; across filtered Nix sources the marker changed between `Cargo.toml`, `build.rs`, and `src/lib.rs`, which made downstream fingerprints stale. The flake normalizes those timestamps:

```
touch -d @0 crates/gammaloop-workspace-hack/Cargo.toml
touch -d @0 crates/gammaloop-workspace-hack/src/lib.rs
touch -d @1 crates/gammaloop-workspace-hack/build.rs
```

One bug in this layer was that scripts passed to Crane's `mkDummySrc` wrote paths such as `crates/foo/src/lib.rs` directly. `mkDummySrc` builds its output under `$out`, so those writes missed the dummy source. Dummy-source edits now write to `$out/...`; post-patch edits in real package builds still use workspace-relative paths.

Dependency-only sources now restore real source directories for already-built workspace dependencies, while keeping the current package's targets dummy. This lets Cargo reuse upstream workspace artifacts in final package builds, and keeps the dependency archive stable across ordinary source edits to the current package.

The Cargo CLI feature set is intentionally narrower than the source/artifact closure. Nix derivations keep resolved workspace dependency sources and artifacts so Cargo can validate fingerprints, but `--features` only names the selected package plus direct workspace dependencies. Cargo rejects qualified features for transitive packages that are not selected and are not direct dependencies. The `65e5d86e` NixCI run failed this way for `crate-deps-gammaloop-tracing-filter` and `crate-deps-spenso-hep-lib`.

What is fixed locally

`crate-spenso` is the clean sentinel for normal target-library reuse because it consumes `linnet` and the workspace hack. After the fixes above:

```
cargo build --profile ci-optim --locked -p gammaloop-workspace-hack -p spenso \
  --features linnet/bincode,linnet/serde,linnet/symbolica,spenso-macros/
shadowing,spenso/shadowing,symbolica/tracing_max_level_info

Compiling spenso v0.6.0
```

The final `crate-spenso-build` phase no longer compiles `linnet` or `gammaloop-workspace-hack`, and it no longer compiles `spenso-macros` in the final package phase. The proc-macro unit is produced by the dependency archive for the consumer context.

The same final-build reuse pattern was observed for narrower consumers before the `gammalooprs` check:

```
crate-idenso-build: compiled idenso only
crate-spenso-hep-lib-build: compiled spenso-hep-lib only
crate-gammaloop-tracing-filter-build: compiled gammaloop-tracing-filter only
```

The two feature-selection failures from the `65e5d86e` NixCI run now build locally. Their dependency artifacts compile the required upstream workspace crates in the consumer context, and their final package derivations compile only the selected package:

```
crate-deps-gammaloop-tracing-filter:
  Compiling gammaloop-tracing-filter
  Compiling gammaloop-tracing-filter-macros
  Compiling linnet
  Compiling spenso

crate-gammaloop-tracing-filter-build:
  Compiling gammaloop-tracing-filter

crate-deps-spenso-hep-lib:
  Compiling spenso-hep-lib
  Compiling linnet
  Compiling spenso
  Compiling idenso

crate-spenso-hep-lib-build:
  Compiling spenso-hep-lib
```

The root `crate-gammalooprs-build` phase is the broadest normal target-library reuse check. After adding the final feature anchors, a local build produced:

```
cargo build --profile ci-optim --locked -p gammaloop-workspace-hack -p gammalooprs \
  --features gammaloop-tracing-filter/clap,gammaloop-tracing-filter/symbolica,idenso/
bincode,idenso/reference-cases,linnet/bincode,linnet/serde,linnet/symbolica,spenso/
shadowing,symbolica/tracing_max_level_info

Compiling gammalooprs v0.3.3
Finished `ci-optim` profile [optimized] target(s) in 1m 39s
```

That final root phase did not compile `vakint`, `linnet`, `spenso`, `idenso`, `spenso-hep-lib`, `gammaloop-tracing-filter`, or `gammaloop-workspace-hack`. Those crates were consumed from earlier workspace artifact derivations.

The source-filter cache boundary was checked with temporary code-only comments:

```
spenso code edit:
  crate-deps-spenso stayed /nix/store/207kj3kab5j87n07i8kims0is6wkv3rq-gammaloop-
crate-spenso-deps-0.1.0.drv
  crate-spenso changed to /nix/store/vs6a6mckr8d9yd8w6r8mjli8c8a5xdsb-gammaloop-
crate-spenso-build-0.1.0.drv

gammalooprs code edit:
  crate-deps-gammalooprs stayed /nix/store/xs8jqfllizzc6bbjyv3qj2li05r7zs2m-
```

```
gammaloop-crate-gammalooprs-deps-0.1.0.drv
  crate-gammalooprs changed to /nix/store/0dmnbsxd0cyqirjxr4vjs7z397sv5pnk-gammaloop-
crate-gammalooprs-build-0.1.0.drv
```

Follow-up audit: workspace crate reuse is context-sensitive

The later `linnest -> linnet` investigation found an additional Cargo constraint: a workspace crate artifact is not universally reusable just because the package name and feature list look the same. Cargo's unit hash also changes with the selected package graph.

Observed locally:

```
crate-linnet-build / standalone linnet context:
  target/ci-optim/.fingerprint/linnet-f122c64f71757531

crate-linnest-deps / linnet as dependency of linnest:
  target/ci-optim/.fingerprint/linnet-58bb076a94f2462f
```

Trying to feed the standalone `crate-linnet` artifact into `crate-linnest` therefore still made Cargo compile `linnet`. The fix is to make `crate-deps-<consumer>` the cache boundary for workspace dependencies in that consumer's graph, and to preserve those workspace dependency artifacts for the final package derivation.

The final package input also has to go through the custom artifact merge step. Crane's normal single-archive unpack path does not recursively materialize the `.prev` chain created by `buildDepsOnlyWithArtifacts`; passing the bare `crate-deps-linnest` archive caused the final package build to lose inherited root artifacts and recompile third-party crates. The final package derivations now receive a self-contained merged archive with the `.prev` chain already materialized before Crane restores it.

The hack timestamp normalization also has to run after unpack in `buildDepsOnlyWithArtifacts`. Running it only while constructing a dummy source is insufficient because Nix store normalization resets file mtimes. Without the post-unpack normalization, Cargo marked `gammaloop-workspace-hack` dirty with:

```
dirty: PrecalculatedComponentsChanged { old: "1.000000000s (src/lib.rs)", new:
"1.000000000s (build.rs)" }
```

That hack rebuild then made dependent workspace crates, such as `linnet`, dirty through `UnitDependencyInfoChanged`.

Current focused verification

The focused `linnest` package path now has this shape:

```
crate-linnet-deps:
  Compiling cgmath
  Compiling linnet

crate-linnest-deps:
  Compiling linnest   # dummy target package
  Compiling linnet    # dependency in the linnest consumer context

crate-linnest-build:
  Compiling linnest   # final real package only
```

The final `crate-linnest-build` phase no longer compiles `linnet`, `gammaloop-workspace-hack`, Symbolica, or the rest of the external graph.

The focused test-binary path for `linnet-py` also no longer recompiles `linnet` in the final test-binary derivation:

```
crate-test-binaries-linnet-py:
  Compiling gammaloop-workspace-hack
  Compiling linnet-py
  Executable unittests src/lib.rs
  Executable unittests src/bin/stubgen.rs
```

This still compiles the current test package and the Hakari anchor in that test context. It does not compile upstream `linnet` again.

The nextest archive layer is now a two-step cache boundary:

```
gammaloop-nextest-binaries-linnet-test-deps:
  cargo test --profile ci-optim --no-run --locked \
    -p clinnet -p gammaloop-workspace-hack -p linnest -p linnet -p linnet-py ...
  Compiling iai-callgrind-runner
  Compiling gammaloop-workspace-hack
  Compiling linnet
  Compiling linnest
  Compiling linnet-py

gammaloop-nextest-binaries-linnet:
  cargo nextest archive ...
  Finished `ci-optim` profile [optimized] target(s) in 0.29s
```

Selecting `gammaloop-workspace-hack` in the nextest package set is required for Symbolica feature anchoring. Without that selected package, the linnet archive path requested a narrower Symbolica unit and rebuilt Symbolica. With the anchor, the test-binary artifact still compiles the selected workspace test units, but it does not compile Symbolica, numerica, or graphica.

Before the later per-package nextest split, the spenso group used the same group-level shape:

```
gammaloop-nextest-binaries-spenso-test-deps:
  cargo test --profile ci-optim --no-run --locked \
    -p gammaloop-workspace-hack -p idenso -p spenso -p spenso-hep-lib \
    -p spenso-macros -p spynso3 ...
  Compiling spenso-macros
  Compiling gammaloop-workspace-hack
  Compiling linnet
  Compiling spenso
  Compiling idenso
  Compiling spenso-hep-lib
  Compiling spynso3

gammaloop-nextest-binaries-spenso:
  cargo nextest archive ...
  Finished `ci-optim` profile [optimized] target(s) in 0.30s
```

An intermediate experiment fed each package's dependency-mode artifact into the nextest group artifact. That was worse: `spynso3`'s dependency setup compiled `linnet`, `spenso`, `idenso`, and `spenso-hep-lib` in a separate consumer context before the spenso test-binary group compiled them again. The nextest groups were later changed again; see "Follow-up audit: nextest package boundaries" below for the current per-package shape and the removal of `spynso3` from the fast `spenso` archive.

The archived nextest runner also has to recreate the source at `/build/source`. The binaries are compiled with `CARGO_MANIFEST_DIR=/build/source/...`; copying the source to the runCommand's default working directory made insta miss snapshots. The runner additionally installs dummy targets for workspace members outside the narrowed nextest source so `cargo metadata` succeeds.

Local runner checks:

```
checks.x86_64-linux.gammaloop-nextest-linnet:
  150 tests run: 150 passed

checks.x86_64-linux.gammaloop-nextest-spenso:
  350 tests run: 350 passed, 27 skipped
```

`checks.x86_64-linux.gammaloop-nextest-core` also runs from the archive, but in the local environment it fails test initialization with `OEM license key not set`. That is a compile-time license input issue, not a cache-reuse or archive layout issue.

Remaining caveats

The root prebuild is intentionally broad and still cold-builds a large unified external graph, including Python/stubgen-related units and `numerica`/`graphica`. This is a cache-root cost, not downstream double work. If that cost is too high, the next optimization should split Python/stubgen prebuilds from the normal Rust test prebuild rather than trying to reuse standalone workspace package artifacts across incompatible Cargo unit contexts.

The nextest archive step is no longer a compile boundary. It receives a group-matched test-binary artifact and only packages fresh binaries.

Feature-context anchors

Before the final fix, `crate-gammalooprs-build` had already stopped recompiling most of the workspace stack, but still compiled `vakint`:

```
gammalooprs
vakint
```

The inherited merged target contained a `vakint` artifact, so this was not a missing Nix edge. Comparing the two `vakint` fingerprints showed that every direct dependency hash matched except `eyre`:

```
vakint archive: eyre dependency hash 1421407417017949601
root archive:   eyre dependency hash 566811204752360852
```

Following the dependency hash down showed that `eyre` was built against a different `indenter` unit in the root graph:

```
vakint archive: indenter features ["default"]
root archive:   indenter features ["default", "std"]
```

`arrayvec` had the same shape earlier: the root graph enabled its normal `default,std` feature set through `vakint`, but the workspace hack did not carry that feature context while `vakint` stayed outside Hakari traversal.

The workspace hack therefore has manual normal-dependency anchors outside Hakari's managed block:

```toml
[dependencies.arrayvec]
version = "0.7"
features = ["std"]
```

```
[dependencies.indenter]
version = "0.3"
features = ["std"]
```

It also has matching build-dependency anchors because Hakari verification checks host-side proc-macro feature contexts separately:

```
[build-dependencies.arrayvec]
version = "0.7"
features = ["std"]

[build-dependencies.indenter]
version = "0.3"
features = ["std"]
```

After those anchors, the `vakint` artifact and root graph request the same third-party units, so the root build reuses `vakint`. The build-dependency anchors also make `cargo hakari verify` pass for the host platform.

Proc-macro packages using Symbolica also need the workspace-hack anchor. Without that, `crate-deps-spenso-macros` inherited the hack artifact but requested a narrower `symbolica` unit and rebuilt `symbolica`, `numerica`, and `graphica`. After allowing proc-macro packages to select the hack anchor, the local `crate-deps-spenso-macros` build was:

```
cargo build --profile ci-optim --locked \
  -p gammaloop-workspace-hack -p spenso-macros \
  --features spenso-macros/shadowing,symbolica/tracing_max_level_info

Compiling spenso-macros v0.3.2
Compiling gammaloop-workspace-hack v0.1.0
Finished `ci-optim` profile [optimized] target(s) in 0.78s
```

It no longer rebuilt the Symbolica stack.

Rejected experiment

Adding `gammalooprs` as a selected feature-context anchor for every dependency crate did make the Cargo commands closer to the final graph:

```
cargo build --profile ci-optim --locked \
  -p gammaloop-workspace-hack -p gammalooprs -p linnet \
  --features gammaloop-tracing-filter/clap,gammaloop-tracing-filter/symbolica,idenso/
bincode,idenso/reference-cases,linnet/bincode,linnet/serde,linnet/symbolica,spenso/
shadowing,symbolica/tracing_max_level_info
```

But it also made every producer compile the dummy `gammalooprs` closure. For example, `crate-linnet-build` compiled `vakint`, `gammalooprs`, `spenso-macros`, `spenso`, `idenso`, `gammaloop-tracing-filter`, and `spenso-hep-lib` in addition to `linnet`. That is worse for CI latency and was backed out.

Follow-up audit: archive and dependency-closure reuse

The later integration archive audit found two more places where the Nix graph looked right but Cargo still rebuilt workspace crates.

Stripped archives must be flattened after Crane installs them

`buildDepsOnlyWithArtifacts` originally stripped dummy workspace artifacts in `postBuild`, then tried to recursively materialize the inherited `target.tar.zst.prev` chain in `postInstall`. That was too early for Crane's `installCargoArtifactsHook`: the hook writes `target.tar.zst` and `.prev`

while `runHook postInstall` is running, after the shell `postInstall` body has already checked for the files.

The flatten-and-strip pass now runs in `preFixup`, after Crane has installed the artifact archive. For stripped dependency archives this leaves a single self-contained `target.tar.zst` and removes `target.tar.zst.prev`.

The strip script also removes unhashed dependency outputs such as:

```
target/ci-optim/deps/libgammaloop_api.rlib
target/ci-optim/deps/libgammaloop_api.so
target/ci-optim/deps/gammaloop_api.d
```

Those files were the source of the stale dummy `gammaloop-api` failure. The merged integration dependency input had previously contained a 15 KiB dummy `libgammaloop_api.so`, so `gammaloop-integration-tests` failed with missing `gammaloop_api::commands`, `session`, and `state`. After stripping unhashed outputs and pre-stripping the current package before package-specific builds, the same merged input contains the real `libgammaloop_api.so` artifact (about 28 MiB locally), and the integration dependency artifact compiles.

Synthetic consumers need artifacts for the same closure they select

The dependency-mode `*-deps-deps` derivations generate a synthetic consumer that depends on the resolved workspace dependency closure. Before this fix, their `cargoArtifacts` input only merged direct dependency artifacts. That mismatch made Cargo rebuild transitive workspace crates in the synthetic consumer layer:

```
gammaloop-crate-gammaloop-tracing-filter-dependency-deps-deps:
  Compiling linnet
  Compiling spenso

gammaloop-crate-idenso-dependency-deps-deps:
  Compiling gammaloop-workspace-hack
  Compiling linnet
  Compiling spenso
```

The merge inputs now use the same resolved dependency closure as the synthetic consumer source. Re-running the same targets locally produced only the synthetic consumer in the deps-of-deps layer:

```
gammaloop-crate-gammaloop-tracing-filter-dependency-deps-deps:
  Compiling gammaloop-ci-consumer-gammaloop-tracing-filter

gammaloop-crate-idenso-dependency-deps-deps:
  Compiling gammaloop-ci-consumer-idenso
```

The real package layers then compiled only their own package plus the synthetic consumer:

```
gammaloop-crate-gammaloop-tracing-filter-dependency-deps:
  Compiling gammaloop-tracing-filter
  Compiling gammaloop-ci-consumer-gammaloop-tracing-filter

gammaloop-crate-idenso-dependency-deps:
  Compiling idenso
  Compiling gammaloop-ci-consumer-idenso
```

The same closure-merge shape was verified for `spenso-hep-lib`, `gammalooprs`, `gammaloop-api`, and `gammaloop-integration-tests`.

Plain integration must not depend on the Python module

The plain integration archive was incorrectly using the same Python environment setup as the Python API archive. That made `gammaloop-nextest-binaries-integration` depend on `gammaloop-python-module`, which in turn rebuilt Python-feature workspace crates before ordinary integration tests could even be archived.

The nextest graph now has a separate `nextestUsesPythonModule` predicate based on the `python-api-tests` feature. The plain integration archive no longer builds `gammaloop-python-module`; the Python module is only an input for the `python-api` archive/check.

Nextest needs a generated feature anchor

The nextest test-binary layer inherited the right per-crate artifacts but still rebuilt upstream workspace crates because its Cargo command only enabled features for selected packages and direct dependencies:

```
cargo test --profile ci-optim --no-run --locked \
  -p gammaloop-integration-tests -p gammaloop-workspace-hack \
  --features spenso/shadowing,symbolica/tracing_max_level_info

Compiling gammaloop-workspace-hack
Compiling linnet
Compiling spenso
Compiling idenso
Compiling gammaloop-tracing-filter
Compiling spenso-hep-lib
Compiling gammalooprs
```

Cargo rejects qualified features for transitive packages unless they are selected or direct dependencies of a selected package. Selecting the full workspace closure would compile too much, so nextest archive builds now add a generated package under `crates/gammaloop-ci-nextest-<group>-features`. The anchor depends on the resolved source package closure with the common feature set and is selected with `-p`. The archive/test commands use `--offline` because the generated path-only workspace member is not in the checked-in lockfile.

With that anchor, the plain integration test-binary layer became:

```
cargo test --profile ci-optim --no-run --offline \
  -p gammaloop-ci-nextest-integration-features \
  -p gammaloop-integration-tests \
  -p gammaloop-workspace-hack \
  --features gammaloop-tracing-filter/clap,gammaloop-tracing-filter/symbolica,...

Compiling gammaloop-workspace-hack
Compiling gammaloop-integration-tests
Compiling gammaloop-ci-nextest-integration-features
Finished `ci-optim` profile [optimized] target(s) in 40.39s
```

The archive layer then compiled only the generated feature anchor and packaged already-built test binaries:

```
gammaloop-nextest-binaries-integration:
  Compiling gammaloop-ci-nextest-integration-features
  Finished `ci-optim` profile [optimized] target(s) in 0.74s
  Archiving 9 binaries ...
```

Local verification:

```
nix build --impure .#checks.x86_64-linux.gammaloop-nextest-binaries-integration --no-
link -L --print-out-paths
  /nix/store/khpd1v0xp2jrbl0x6ida5275yxyavda1-gammaloop-nextest-binaries-
integration-0.1.0

cached rerun:
  /nix/store/khpd1v0xp2jrbl0x6ida5275yxyavda1-gammaloop-nextest-binaries-
integration-0.1.0
```

The Python API archive has the same nextest shape after the separate Python module build:

```
gammaloop-nextest-binaries-python-api-test-deps:
  Compiling gammaloop-integration-tests
  Compiling gammaloop-workspace-hack
  Compiling gammaloop-ci-nextest-python-api-features

gammaloop-nextest-binaries-python-api:
  Compiling gammaloop-ci-nextest-python-api-features
  Finished `ci-optim` profile [optimized] target(s) in 0.72s
  Archiving 1 binary ...
```

Local verification:

```
nix build --impure .#checks.x86_64-linux.gammaloop-nextest-binaries-python-api --no-
link -L --print-out-paths
  /nix/store/hiv5ils5196i8pis4697353sjzh2bci0-gammaloop-nextest-binaries-python-
api-0.1.0

cached rerun:
  /nix/store/hiv5ils5196i8pis4697353sjzh2bci0-gammaloop-nextest-binaries-python-
api-0.1.0
```

Python remains a separate feature family

The Python module path cannot reuse the normal `gammalooprs` and `gammaloop-tracing-filter` artifacts byte-for-byte. Enabling `gammaloop-api/python_abi` and `gammaloop-api/pyo3-extension-module` activates Python/PyO3 features on those crates, so Cargo requests different units.

Observed local Python dependency layer:

```
gammaloop-api-python-deps:
  Compiling gammalooprs
  Compiling pyo3-build-config
  Compiling pyo3-ffi
  Compiling pyo3-macros-backend
  Compiling pyo3
  Compiling numpy
  Compiling gammaloop-api      # dummy target, stripped from the dependency artifact
  Compiling pyo3-macros
  Compiling gammaloop-tracing-filter
```

The following Python build/package/archive layers reused that Python-feature artifact family: the real Python build compiled only `gammaloop-api`, the package install did not print a `Compiling ...` line, and nextest compiled only the Python API test package plus the anchor. If this remaining Python-feature workspace compile is too coarse, the follow-up is to split Python-feature per-crate artifacts rather than moving PyO3 features into the common prebuild.

Historical audit: nextest package boundaries (superseded)

The package-boundary and strict-reuse sections below describe the former mixed `crate-test-support-*` design. They are retained as investigation history and are superseded by Source and test-artifact ownership split.

The nextest archive graph was changed from one test-binary derivation per group to one test-binary derivation per package, with the final group archive merging those package artifacts. This makes the Nix graph expose package-sized cache boundaries instead of hiding all test compilation inside `gammaloop-nextest-binaries-<group>-test-deps`.

The checked Guppy graph now also records workspace dependency edge features:

```
"spynso3": {
  "spenso": [
    "python",
    "shadowing"
  ]
}
```

Those incoming features are folded into the common Crane feature set, except for Python wrapper packages that are intentionally outside the fast Rust cache root:

```
workspaceFeatureUnificationExcludedPackages = [
  "linnet-py"
  "spynso3"
];
```

`spynso3` was removed from the fast `spenso` nextest archive. Keeping it there forces `spenso/python` and `symbolica/python_export` into the same resolver context as normal Rust `spenso` tests. A local experiment added `symbolica/python_export` to the workspace-hack anchor; that made the hack producer cold-build PyO3/numpy/stubgen-related Symbolica dependencies and still did not stop `spynso3` from rebuilding workspace crates because PyO3 features were not anchored. The measured cold hack producer was:

```
gammaloop-crate-gammaloop-workspace-hack-deps:
  cargo build ... --features symbolica/python_export,symbolica/tracing_max_level_info
  Finished in 5m33s
  target archive: 818 MiB -> 218 MiB compressed
```

That route was backed out. The next step for `spynso3`, if it must be tested in NixCI, should be a separate Python-wrapper graph with its own PyO3/Symbolica feature family. Putting it back into the fast `spenso` graph would either reintroduce workspace recompilation or make the root prebuild much heavier.

Local verification after the per-package nextest split:

```
nix build --impure .#checks.x86_64-linux.gammaloop-guppy-workspace-graph --no-link -L
--print-out-paths
  /nix/store/lkqg567a9ig75jjh01l6yh7wqfkvx1fq-gammaloop-guppy-workspace-graph-check

nix build --impure .#checks.x86_64-linux.gammaloop-nextest-binaries-spenso --no-link
-L --print-out-paths
  /nix/store/k60aaha4crsvnk1l570lr963w0yaqr4q-gammaloop-nextest-binaries-spenso-0.1.0

nix build --impure .#checks.x86_64-linux.gammaloop-nextest-binaries-integration --no-
link -L --print-out-paths
  /nix/store/0imvvji66p0ww16xzlqfr60pbwlfvahm-gammaloop-nextest-binaries-
```

```
integration-0.1.0

nix build --impure .#checks.x86_64-linux.gammaloop-nextest-binaries-python-api --no-
link -L --print-out-paths
  /nix/store/2jyb556w3za039kryg6rg30h534x693l-gammaloop-nextest-binaries-python-
api-0.1.0
```

Observed compile shape:

```
gammaloop-nextest-binaries-integration-gammaloop-integration-tests-test-deps:
  Compiling gammaloop-workspace-hack
  Compiling gammaloop-integration-tests
  Compiling gammaloop-ci-nextest-integration-gammaloop-integration-tests-features

gammaloop-nextest-binaries-python-api-gammaloop-integration-tests-test-deps:
  Compiling gammaloop-integration-tests
  Compiling gammaloop-workspace-hack
  Compiling gammaloop-ci-nextest-python-api-gammaloop-integration-tests-features

gammaloop-nextest-binaries-spenso:
  Compiling gammaloop-ci-nextest-spenso-features
  Archiving 14 binaries
```

So the final archive layers are no longer compile boundaries; they only compile their generated feature anchor and package already-built binaries.

The per-package nextest prebuilds now consume shared test-support artifacts instead of normal package artifacts. This matters because `cargo build` and `cargo test --no-run` produce different Cargo units even under the same named profile. A normal `crate-deps-*` artifact cannot satisfy test-harness units.

The public `crate-test-binaries-*` package attrs also alias the shared test-support artifact for that package's test SCC. For example, these two evals now return the same derivation:

```
nix eval --impure --raw .#packages.x86_64-linux.crate-test-binaries-spenso-
macros.drvPath
nix eval --impure --raw .#packages.x86_64-linux.crate-test-support-spenso.drvPath
  /nix/store/d3lvg9h3pa9yl4yi6pxrcwf7gsiny5my-gammaloop-crate-test-support-spenso-
deps-0.1.0.drv
```

That removes the separate exposed per-package test-binary build that could make NixCI schedule package-specific duplicate work.

The downstream nextest package prebuilds are now clean. They inherit `crate-test-support-spenso`, then compile only their generated feature-anchor package before archiving/running existing binaries:

```
gammaloop-nextest-binaries-spenso-spenso-macros-test-deps:
  cargo test --profile ci-optim --no-run --offline \
    -p gammaloop-ci-nextest-spenso-spenso-macros-features \
    -p gammaloop-workspace-hack -p spenso-macros ...
  Compiling gammaloop-ci-nextest-spenso-spenso-macros-features
  Finished in 0.74s

gammaloop-nextest-binaries-spenso-spenso-hep-lib-test-deps:
  Compiling gammaloop-ci-nextest-spenso-spenso-hep-lib-features
  Finished in 0.70s
```

```
gammaloop-nextest-binaries-spenso-idenso-test-deps:
  Compiling gammaloop-ci-nextest-spenso-idenso-features
  Finished in 0.75s

gammaloop-nextest-binaries-spenso-spenso-test-deps:
  Compiling gammaloop-ci-nextest-spenso-spenso-features
  Finished in 0.72s
```

The final archive layer remains clean:

```
gammaloop-nextest-binaries-spenso:
  Compiling gammaloop-ci-nextest-spenso-features
  Finished in 0.66s
  Archiving 14 binaries
```

The next workspace-crate recompilation failure was one layer earlier, between shared test-support SCCs. Before the synthetic-consumer fix, `crate-test-support-linnest` compiled `linnet` for the `linnet/linnest` test component:

```
crate-test-support-linnest:
  cargo test --profile ci-optim --no-run --offline \
    -p gammaloop-ci-test-support-linnest-features \
    -p gammaloop-workspace-hack -p linnest -p linnet ...
  Compiling gammaloop-workspace-hack
  Compiling linnet
  Compiling linnest
  Compiling gammaloop-ci-test-support-linnest-features
```

Then `crate-test-support-spenso`, which depended on that artifact, still compiled `linnet` again in the `spenso/spenso-macros` test component:

```
crate-test-support-spenso:
  cargo test --profile ci-optim --no-run --offline \
    -p gammaloop-ci-test-support-spenso-features \
    -p gammaloop-workspace-hack -p spenso -p spenso-macros ...
  Compiling spenso-macros
  Compiling linnet
  Compiling spenso
  Compiling gammaloop-ci-test-support-spenso-features
```

That was a failure under the strict "no workspace crate recompilation" goal. It was no longer hidden downstream in package/archive derivations, but the fine-grained test-support SCC graph still asked Cargo for a different enough test context that the inherited `linnet` unit was not reused.

A focused fingerprint check showed that the two `linnet` units have the same visible `linnet` features:

```
["bincode", "default", "drawing", "nodestore-vec", "serde", "symbolica"]
```

One difference was the `cgmath` dependency unit: the `linnest` support context enabled `cgmath/serde`, while the `spenso` support context reached `cgmath` through `linnet/drawing`. A temporary manual `cgmath/serde` workspace-hack anchor was tested and then backed out. It made the common root broader but `crate-test-support-spenso` still compiled `linnet`, so `cgmath` feature mismatch was not the whole cause. The manifest-side fix is to make `linnet/drawing` consistently enable `cgmath/serde`, which removes the separate no-`serde` `cgmath` unit.

The remaining difference was Cargo context: `spenso` wanted a `lib-linnet` unit for "dependency of a selected test package", while the earlier `linnest` support derivation had only built the standalone/

component context. The support graph now prebuilds dummy next-consumer contexts and strips those dummy consumer artifacts afterward. This keeps downstream source code out of the upstream support source hash, but gives Cargo the exact upstream workspace units later components request.

Local verification after narrowing the synthetic consumers:

```
crate-test-support-linnest:
  cargo test ... \
    -p gammaloop-ci-test-support-linnest-features \
    -p gammaloop-workspace-hack -p linnest -p linnet -p spenso -p spenso-macros ...
  Compiling linnet
  Compiling spenso-macros
  Compiling spenso
  Compiling linnest
  target archive: 220 MiB => 66.4 MiB

crate-test-support-spenso:
  cargo test ... \
    -p gammaloop-ci-test-support-spenso-features \
    -p gammaloop-tracing-filter -p gammaloop-workspace-hack -p idenso -p spenso -p
spenso-macros ...
  Compiling spenso-macros
  Compiling spenso
  Compiling idenso
  Compiling gammaloop-tracing-filter
  no Compiling linnet
  target archive: 136 MiB => 42.4 MiB

crate-test-support-idenso:
  cargo test ... \
    -p gammaloop-ci-test-support-idenso-features \
    -p gammaloop-workspace-hack -p idenso -p spenso-hep-lib ...
  Compiling idenso
  Compiling spenso-hep-lib
  no Compiling spenso
  no Compiling linnet
```

The exported support archives were checked to ensure dummy consumer artifacts are stripped. For example, `crate-test-support-spenso` exports `linnet`, `spenso`, and `spenso-macros` fingerprints, but not `idenso` or `gammaloop-tracing-filter` fingerprints.

The current conclusion is:

- Normal package outputs reuse workspace crate artifacts in their final package layer.
- Test-support SCCs prebuild the next dummy consumer context, so later support SCCs reuse upstream workspace crates instead of recompiling them.
- Downstream nextest package prebuilds and archive layers reuse the shared test-support artifacts and no longer recompile workspace crates.
- The tradeoff is a larger earlier support artifact. The `linnest` support archive grew from roughly `152 MiB => 45 MiB` to `220 MiB => 66 MiB`, but the work moved to a NixCI-cacheable dependency layer.

The previous failed shape is kept here for comparison. Feeding `spenso`'s normal package artifact into `spenso-macros` was insufficient because Cargo rebuilt `linnet` and `spenso` in the `spenso-macros` test context:

```
gammaloop-nextest-binaries-spenso-spenso-macros-test-deps:
  Compiling gammaloop-workspace-hack
  Compiling linnet
  Compiling gammaloop-ci-nextest-spenso-spenso-macros-features
  Compiling spenso-macros
  Compiling spenso
```

A follow-up experiment added a test-specific dependency producer with the same generated anchor name. That producer compiled `linnet`, `spenso`, and `spenso-macros`, but the final `spenso-macros` test derivation still compiled the same workspace units again. This was backed out because it added work without improving reuse. The current hypothesis is that Cargo's test-unit fingerprint includes enough context from the real current package graph that the dummy/dependency producer cannot satisfy the final package test request.

Follow-up audit: package artifacts preserve workspace crates

The `65e5d86e` NixCI run failed before the nextest archive nodes could prove anything about reuse. The hard failures were:

```
packages.x86_64-linux.crate-deps-gammaloop-tracing-filter
packages.x86_64-linux.crate-deps-spenso-hep-lib
```

Both failed because the generated command passed transitive package feature flags to a package that did not directly own those feature names:

```
cargo build ... -p spenso-hep-lib \
  --features idenso/bincode,idenso/reference-cases,linnet/bincode,linnet/
serde,linnet/symbolica,...

error: the package 'spenso-hep-lib' does not contain these features:
  linnet/bincode, linnet/serde, linnet/symbolica
```

The deeper bug was that public `crate-deps-*` outputs used the older dummy-current-package artifact path. That path compiled with the current package target dummied and then stripped the current package artifact, so a downstream package received dependency archives but not the compiled workspace crate it needed to reuse. The public `crate-deps-*` outputs now point at the dependency-mode artifacts instead. Those build a generated consumer package and preserve the real package artifact.

The dependency-mode artifacts now also preserve the package's resolved workspace dependencies. Without that, the final `crate-*` package derivation received the current package artifact but not upstream workspace crates, and Cargo rebuilt `linnet`/`spenso` in the final package build.

Final package commands now request the full resolved workspace feature set and select the non-proc-macro feature owners. Proc-macro crates are kept as dependencies, not selected roots, because selecting `spenso-macros` as a root created a separate Cargo unit and rebuilt it.

Local verification after this change:

```
nix build --impure .#packages.x86_64-linux.crate-gammalooprs --no-link -L --print-
out-paths
  crate-deps layer:
    Compiling gammalooprs
    Compiling gammaloop-ci-consumer-gammalooprs
  final crate layer:
    Finished in 0.32s, no Compiling lines

nix build --impure .#packages.x86_64-linux.crate-spenso-hep-lib --no-link -L --print-
```

```
out-paths
  crate-deps layer:
    Compiling spenso-hep-lib
    Compiling gammaloop-ci-consumer-spenso-hep-lib
  final crate layer:
    Finished in 0.29s, no Compiling lines

nix build --impure .#packages.x86_64-linux.crate-gammaloop-tracing-filter --no-link -
L --print-out-paths
  final crate layer:
    cargo build ... -p gammaloop-tracing-filter -p gammaloop-workspace-hack -p linnet
-p spenso ...
    Finished in 0.29s, no Compiling lines

nix build --impure .#checks.x86_64-linux.gammaloop-guppy-workspace-graph --no-link --
print-out-paths
  /nix/store/lkqg567a9ig75jjh01l6yh7wqfkvx1fq-gammaloop-guppy-workspace-graph-check
```

Historically, the remaining double work was in the shared test-support SCCs: `crate-test-support-spenso` compiled `linnet` after inheriting `crate-test-support-linnest`. Synthetic next-consumer contexts now move that work into the earlier cacheable support layer. The former public `crate-test-binaries-*` attrs pointed at those shared support artifacts, so they did not add another per-package test-prebuild layer on top of that.

Historical strict reuse audit (superseded)

The success criterion for that audit was stricter than "Nix derivations are ordered correctly": once a workspace crate had been compiled in the cacheable support boundary, later package-specific test-prebuilds and nextest archive derivations must reuse that artifact. Recompiling a generated feature-anchor crate is allowed; recompiling `linnet`, `spenso`, `idenso`, `spenso-hep-lib`, `gammalooprs`, `gammaloop-api`, or similar real workspace crates downstream is a failure.

Two details were needed after the package-artifact fix:

- `clinnet` is its own nextest group. It has no `symbolica` dependency, but the old grouped `linnet` archive selected `clinnet` together with `linnet`/`linnest`/`linnet-py`. That broader group changed Cargo fingerprints through shared dependencies such as `indicatif`, so the final archive rebuilt `clinnet`.
- Test support builds binary targets only for the real component packages in that support SCC. Selecting binaries from synthetic consumers made support layers such as `crate-test-support-gammalooprs` build the `gammaloop-api` binary, which moved high-level work into a lower-level cache boundary.

The aggregate local check now completes:

```
nix build --impure .#checks.x86_64-linux.gammaloop-nextest-binaries --no-link -L --
print-out-paths
  /nix/store/x7ymfpn5j4q2nhxjhfk2c911bfikp7hj-gammaloop-nextest-binaries
```

During that run, changed package-specific prebuilds compiled only generated anchor crates. Examples:

```
gammaloop-nextest-binaries-spenso-spenso-test-deps:
  Compiling gammaloop-ci-nextest-spenso-spenso-features
  no Compiling linnet
  no Compiling spenso
```

```
gammaloop-nextest-binaries-linnet-linnet-test-deps:
  Compiling gammaloop-ci-nextest-linnet-linnet-features
  no Compiling linnet

gammaloop-nextest-binaries-core-gammalooprs-test-deps:
  Compiling gammaloop-ci-nextest-core-gammalooprs-features
  no Compiling gammalooprs
  no Compiling gammaloop-api
```

The final archive derivations also compiled only their generated group anchors:

```
gammaloop-nextest-binaries-clinnet:
  Compiling gammaloop-ci-nextest-clinnet-features

gammaloop-nextest-binaries-linnet:
  Compiling gammaloop-ci-nextest-linnet-features

gammaloop-nextest-binaries-spenso:
  Compiling gammaloop-ci-nextest-spenso-features

gammaloop-nextest-binaries-vakint:
  Compiling gammaloop-ci-nextest-vakint-features

gammaloop-nextest-binaries-core:
  Compiling gammaloop-ci-nextest-core-features
```

Normal integration and Python API remain distinct checks. Normal integration no longer enables `python-api-tests` and reuses the ordinary Rust support artifacts. The Python API check intentionally compiles `gammaloop-integration-tests` once with `python-api-tests`, and the Python module path intentionally compiles a separate PyO3 feature family for `gammaloop-api` and its dependents. That is not reusable with the normal Rust feature family.

Source and test-artifact ownership split

The test cache boundary now follows source and artifact ownership instead of recursively carrying dependency test units:

```
third-party dependencies
  -> resolved-feature library support
  -> package-local test binaries
  -> fixture-free nextest archive
  -> nextest run + runtime fixtures
```

Each workspace package has two source tiers. The production tier contains the workspace manifests and lockfile, build scripts, library and proc-macro sources, and declared compile-time assets. It excludes Cargo test, bench, example, and binary targets, as well as separate `cfg(test)` modules. The own-test tier adds those sources back only for the package being tested. Inline `#[cfg(test)]` blocks remain part of their containing production `.rs` file. Cargo still validates explicitly declared target paths during library builds, so absent test, bench, example, and binary entrypoints are represented by no-op dummy files rather than by the dependency's real sources.

`crate-test-dependencies-<component>` is keyed by the sorted resolved workspace feature vector, target, CI profile, and compile environment. It runs only `cargo build ... --lib`. A generated library anchor models direct dev dependencies and the existing dev-dependency SCCs, so build scripts, host proc macros, and resolver-v2 dependency variants are ready without compiling any workspace test target. Proc-macro SCCs additionally use generated target and host library facades: the target facade mirrors the proc macro's normal and dev dependency declarations, while the host

facade preserves Cargo's proc-macro compilation context. Both still run only `cargo build --lib`. Identical contexts share the same derivation; the Python API feature context remains distinct from ordinary integration tests.

`crate-test-binaries-<package>` is now a genuine package-local derivation. It consumes that context's dependency artifact, receives the own-test source for that package alone, and runs `cargo test --no-run -p <package>`. Each nextest archive consumes its matching package-local binary artifact directly. Dependency test sources and test binaries therefore never cross a downstream package boundary.

The own-test tier is split again at the archive boundary. Compilation and packaging receive Rust test, bench, example, and binary sources plus explicitly declared compile-time assets. The run derivation adds package-local fixture paths, snapshots, shared CLI examples, and other runtime data. Consequently a fixture edit changes the owning run check without rebuilding or repackaging its test binaries.

The older test-support and synthetic-consumer measurements above are retained as the history that led to this split; their mixed `crate-test-support-*` artifact shape is no longer the current implementation.

The ownership boundary was checked with isolated source-tree mutations and `nix eval ...drvPath` comparisons:

Mutation	Changed drvPaths	Byte-identical drvPaths
`vakint/tests/*`	Vakint test binary and archive	Vakint support; integration support, binary, and archive
`vakint/src/*`	Vakint support and binary; integration support, binary, and archive	Linnet and Clinnet test binaries
Gammalooprs test source or compile-time asset	Gammalooprs test binary and core archive	Gammalooprs support; integration support, binary, and archive
Python-only integration feature	Python API archive	Ordinary integration binary and archive
Integration-test `build.rs`	Integration support and binary	Global third-party artifacts; Vakint support and binary
Integration runtime fixture	Integration run check	Integration test binary and archive; core and Python API run checks

`nix why-depends --derivation` also confirms that the integration test binary reaches Vakint's library-support derivation but does not depend on Vakint's test-binary derivation. The pre- and post-split Vakint archives both list the same 9 suites and 74 tests.

Follow-up audit: crate-level check splits

The per-crate package and nextest cache graph should not be copied directly to every Cargo mode. I tested crate-level `clippy`, `doc`, and doctest splits and rejected them because the actual logs showed downstream rebuilds of already compiled dependency work.

The `clippy` experiment used a dependency-mode layer first, then a final real package lint layer. The dependency-mode nodes improved reuse, but linting the real selected package still changed Cargo's unit fingerprints and rebuilt the expensive dependency graph:

```
crate-clippy-deps-spenso:
  no Checking symbolica
  no Checking linnet

crate-clippy-gammalooprs:
  Compiling symbolica
  Checking linnet
  Checking spenso
  Checking idenso
```

The `doc` experiment failed in the same way. A narrow package could sometimes reuse the inputs, but another package in the same dependency family re-entered the Symbolica graph:

```
crate-doc-linnet:
  no Checking symbolica

crate-doc-spenso-macros:
  Checking numerica
  Checking graphica
  Checking symbolica
```

Even the single workspace doc artifact fed from the merged workspace check artifact does doc-mode work that is not reusable from the normal check/test artifacts:

```
cargo doc --profile ci-optim --locked --workspace ... --no-deps
  Compiling rug
  Compiling numerica
  Checking graphica
  Checking symbolica
```

The doctest split looked viable for one component, but failed the broader representative path. The `spenso` component reused cleanly:

```
crate-doctest-spenso:
  cargo test --profile ci-optim --doc ... -p spenso -p spenso-macros ...
  Finished in 0.29s
  no Compiling symbolica
  no Compiling linnet
  no Compiling spenso
```

When the `gammalooprs` doctest path pulled in downstream components, the split either produced illegal transitive `pkg/feature` flags or, after narrowing those flags to direct packages, recompiled real workspace crates:

```
crate-doctest-gammaloop-tracing-filter:
  Compiling linnet
  Compiling spenso
  Compiling gammaloop-tracing-filter

crate-doctest-spenso-hep-lib:
  Compiling linnet
  Compiling spenso
  Compiling idenso
  Compiling spenso-hep-lib
```

The workspace aggregate artifact approach also proved too expensive as a cache object. In the `000c330d7caae29bf4d4fd7136e4929c5e6f1f19` NixCI run, `workspaceCheckCargoArtifacts`, `workspaceClippyCargoArtifacts`, and `workspaceDocCargoArtifacts` spent minutes on cache download/upload and compression even when they were already cached. They are invalidated by ordinary workspace crate edits, and no later high-value test step consumes their exact Cargo mode output. The replacement is a terminal `gammaloop-check` job that runs `cargo check --all-targets` and writes only a tiny success marker.

So the current rule is: only expose crate-level CI attrs for a Cargo mode after the persisted logs prove that downstream nodes do not compile real workspace crates or the Symbolica/numerica/graphica graph. For now, crate-level reuse is kept for package and nextest/test-binary artifacts. Workspace `check`, `clippy`, `doc`, and doctest are terminal checks rather than misleading per-crate DAGs or reusable artifact producers. They consume the base dependency artifact to avoid starting completely cold, but they do not publish merged workspace target trees back to the cache.

Documentation publishing uses one reusable real-workspace Cargo artifact producer seeded with the native GammaLoop API library archive. It retains the generic catalogue exporter, each isolated Python inventory exporter, the content-test dependency contexts, the documentation builder, Rustdoc, and a combined GammaLoop/Vakint reference-catalogue context. Stable dummy content primes the test dependencies without admitting prose into the producer key; the terminal derivation still compiles and runs the real content-sensitive test targets. The combined context is last because Cargo retains only one active feature fingerprint for each workspace unit and it is also the terminal consumer's first context. The Python inventories remain separate because PyO3 stub registration uses a process-wide inventory. The producer enumerates the exporter crate’s non-default Cargo features once each, rather than product identifiers: Idenso shares Spenso’s Python binding owner. The terminal consumer builds the combined Community wheel; there is no separate standalone Linnet extension build in the reusable producer. The catalogue helper enables its native feature anchor only for reference generation, so its use as a build dependency does not pull GammaLoop into a separate host-library build.

The terminal source set also carries Spenso’s packaged tensor stub alongside its documentation snapshot, so the isolated exporter validates both copies. FeynKit’s amplitude Rust reference is included in the terminal route assertions; its compile-time tests receive the existing model fixture directory through the per-package extra-source map. These inputs extend their owning package and documentation closures without changing the reusable producer partition.

The producer is keyed only by Cargo manifests, Rust, Cargo configuration, and non-Cargo build inputs. It and its consumer use the same relative Cargo target, compile-time Symbolica setting and workspace-hack timestamp normalization. Native packages, test harnesses and documentation share the optimized `ci-optim` profile, compatible features and Python interpreter at compile time. Python extensions retain their separate ABI settings, and Rustdoc metadata still needs its own compilation. Documentation assets, Typst, CSS, prose, publication-catalogue, channel, tag, and commit metadata do not change the reusable Cargo artifact. Checks whose Rust source is generated from the manuals remain in the terminal Pages derivation so they still validate the content being published.

The terminal documentation derivation builds the combined Community wheel from `examples/notebooks/symbolica-host/Cargo.toml` and installs it with `--offline --no-deps` into an isolated Python environment before exercising the Linnet Python surface. The host owns the single Symbolica extension. Cargo and uv run offline; runtime dependencies must already be staged in the terminal closure. Optional Python Typst compilation for vector export is separate from native notebook rendering and source-only Typst project export. The authored documentation renderer receives its package tree from `docsTypst` for MiTeX and other page assets. Embedded inputs enter

only their owning package's producer source key; Python runtime dependencies remain in the terminal closure.

Drawing package inputs track the current Kurvst and Linnest plugins and their MIT, Clarabel and NetworkX license files, replacing the retired ec-layout plugin inputs. Clinnet also receives its mark and layout fixtures and the portal theme used by its render tests. Linnet's runtime corpus input is limited to DOT files in the declared graph roots, alongside its existing nauty resources. The Clinnet nextest group receives `docsTypst` through `TYPST_TEST_EXECUTABLE`, including stock CeTZ 0.5.2, so the Typst and native/Wasm mark checks run as required tests. The per-package input and runtime-tool owners retain this separation.

Community rendering and UFO runtime inputs
The shared Python rendering package is `linnet-render-py`, using Linnest's native SVG renderer. The graph API remains in `linnet-py`, and both packages retain separate Cargo dependency boundaries. No current build reads the Typst packages under `vendor/typst-packages`; the authored documentation renderer receives its package tree from `docsTypst`. Asset manifests accompany their owning packages in dependency and production builds. `linnet-render-py` owns `data.json`, the checked-in browser font and the shared `view.py`/`view.js` measurement widget. Spynso retains its tensor Typst export sources, while Linnet's graph adapter retains its subgraph styling. FeynKit's native notebook widgets also own their embedded CSS, JavaScript and HTML inputs. Test producers explicitly stage their embedded Python, Atom, JSON and graph fixtures without adding those inputs to production packages. The `gammaloop-render-data` runtime package copies the checked-in MathML font and its license using the `linnet-render-py` asset manifest. The documentation test closure also stages the shared browser font and its license from `linnet-render-py/fonts`. Renderers retain their compiled hash checks. Only rendering test groups, doctests and documentation receive this resource closure; it remains separate from compiled producers. The development shell adds it to `SYMBOLICA_DATA_DIRECTORY`, preserving any additional directories already configured by the user. UFO import tests use the released wheels locked in `nix/ufo-runtime/uv.lock` (Symbolica 3.0.0 and ufo-model-loader 1.0.1). This runtime lock is independent of the root environment's standalone Symbolica pin and the combined Community host's Cargo source pins. Native Rust continues to use the workspace Cargo lock. The UFO wheel closure is attached only to the selected runtime groups and does not enter unrelated compiled producers.

Follow-up: persistent Typst worlds in the live watcher

The release and Pages builders still use the pinned Typst 0.15 command-line renderer. That path remains the reproducible publishing reference and does not link the Typst compiler into the normal documentation Cargo artifact. The local `docs-watch` path enables the optional `watch` feature, which owns the file watcher, live HTTP server, and embedded Typst compiler. Ordinary builds omit those dependencies and the watch command. The compiler stays in the watcher process. Typst's source-independent feature notices are emitted once per watcher; source warnings and errors remain visible on each compilation. Non-blocking developer-note maintenance reminders are reported by `docs-check`, while every build still enforces registry validity. Verified-scope content drift is also reported as a warning during `docs-watch`, so an architectural edit does not block its own preview. Ordinary checks and site builds retain the review gate; unreadable sources and invalid scope boundaries still fail in watch mode.

The verified Nix scopes select the documentation source filters, reusable producer inputs and commands, per-package source ownership, and terminal render/test consumers. Each range has a unique start and exclusive end anchor in `docs/developers.toml`; development-shell package

additions outside those ranges do not require a cache-architecture attestation. Broader review triggers remain recorded for changes to the surrounding build graph.

The watcher now owns one stable Typst entrypoint and `World` for each selected project, plus a developer-notes world for an all-project build. It no longer starts a fresh Cargo build worker or a fresh `typst compile` process for each generation. Product registries, portal data, talks, publications, developer metadata, and the generated API root are still loaded again for every generation, so retaining compiler state does not retain stale site configuration.

Each world keeps its `FileStore`, font store, library inputs, package loader, and fixed build time. After a compilation, `FileStore::reset` marks loaded files stale. The next access reloads them and updates reusable Typst `Source` values in place; it does not literally reset only the files named by the file watcher. This source identity, together with the process-global `comemo` cache, is what makes the next compilation incremental. `comemo::evict(10)` runs once per complete site generation rather than once per product, so an all-project edit does not age six worlds' cache entries six times.

Bundle output remains generation-local and publication remains atomic. Each world writes the same sorted, NUL-delimited dependency description used by the existing source watcher. A successful generation replaces the watched set; a failed partial all-project generation extends the last complete set and leaves the last successfully published site in place. Changes to the running builder or documentation-schema Rust code exit with an explicit restart request, because an in-process watcher cannot hot-recompile its own implementation.

The watch session also retains one complete Rustdoc sidecar per project. The first successful generation creates those sidecars; a Typst-only edit copies them into the new generation without deleting `target/doc` or launching `cargo doc`. Rust source, crate inputs, Rustdoc styling, and the registered project/component configuration mark the sidecars dirty. Cargo configuration changes restart the watcher so its process environment cannot become stale. The sidecars remain dirty until a complete generation succeeds, so a failed Rustdoc refresh cannot make an older sidecar appear current. New sidecars replace their prior cache atomically only after all registered root-crate indexes and Rustdoc's shared source, search, and static resources exist.

Component API catalogues follow the same watch-session model. The first generation exports and validates every registered Rust and Python catalogue, then atomically publishes that complete set to a session cache. Later Typst-only generations still run the full documentation checker, but the checker parses the cached catalogues instead of starting one Cargo exporter per component; the product build copies the same checked artifacts rather than exporting them a second time. Rust and other crate inputs, checked-in Python stubs, Cargo inputs, and the product registry mark the catalogue cache dirty. A failed refresh leaves the previous cache intact and keeps the cache dirty until a complete generation succeeds. Release, Pages, and one-shot builds use generation-local catalogues, so no session artifact can enter a published build without being regenerated there.

The live watcher also retains the fully generated and decorated API-reference subtree for each project. A Typst-only generation restores those pages, then recompiles and redecorates every authored page; authored routes below `reference/` therefore remain live inputs rather than cached output. Search is rebuilt from the combined tree and the complete link validator still runs before atomic publication. Crate, Cargo, checked-in API, Rustdoc-style, and project-registry changes regenerate the reference subtree. Missing, partial, or build-metadata-mismatched cache entries fall back to generation, and a refreshed tree replaces its predecessor only after all declared generated routes exist.

This boundary is deliberately different from "prose never recompiles." Typst still runs `typst::compile` after an edit and uses retained source identities and memoized queries to redo only

affected work. Rust documentation comments change item pages, and ordinary Rust comments change Rustdoc's rendered source pages, so either kind of Rust-source prose correctly invalidates the Rustdoc sidecar. Only prose owned by Typst bypasses Rustdoc completely.

The watcher profile optimizes third-party dependencies at level 2, matching the reason Typst's own documentation watcher uses an optimized development profile. Nix exposes the package tree from `typst.withPackages` through `TYPST_PACKAGE_CACHE_PATH`; without this explicit environment variable, an embedded world would bypass the executable wrapper that normally injects the CeTZ, MiTeX, and Tidy packages.

The dedicated `alphal00p-docs-watch` flake check builds the feature-enabled documentation builder and its schema dependency. It checks all targets, runs the persistent reload and failure-recovery tests together with a representative CLI byte-parity test, exercises the filesystem watcher and live HTTP server, and applies Clippy with warnings denied. Keeping this separate from the normal workspace and Pages artifacts tests the opt-in path without adding the watcher, server, or Typst compiler dependencies to release builds.

The final Pages derivation remains terminal. Its identity includes all rendered documentation sources, the publication channel and optional snapshot tag, and the documented commit and timestamp. An exact rerun can substitute the complete static documentation site, while a content change rebuilds it on top of the stable workspace archive. The Pages workflow and the Nix producer workflow build the same reusable Nix artifact chain in separate Hestia cache namespaces. Hestia v3 action roots are scoped only by ref and system, so sharing a namespace across independent workflows would let either workflow replace the other's roots. Each workflow reuses the chain across its own runs; the Nix workflow realizes the raw documentation artifact before its consumer matrix is released.

Rustdoc can emit unresolved pseudo-links inherited from dependency prose, such as Rust paths that are not navigable in a crate-only sidecar. Each copied sidecar now runs the same link normalization used by whole-site validation before an immutable snapshot is compared. The final validator still scans the combined site. This ordering keeps a second build byte-identical to the first: the previously published snapshot and the fresh candidate are both already in their normalized form when immutability is checked.

Revisions with live notebook support add a publication step after the Nix build. The Pages workflow copies the static site into writable staging, then exports Marimo notebook assets with a combined Symbolica Community WebAssembly wheel built from the documented revision. That wheel has a separate cache keyed by the exact source commit and pinned cibuildwheel version; the export uses pinned Marimo and the source commit's timestamp. This step leaves the Nix Cargo producer boundary unchanged. Notebook assets live inside the selected product version and participate in immutable snapshot comparisons. Historical revisions without notebook support keep the original static bundle.

The physics notebook export also runs the native GammaLoop CLI to import the bundled Standard Model and `save dot` into temporary staging. Its generated particle map and canonical drawing packages become `public/gammaloop-drawing.zip`. Browser Python passes edited DOT and controls to those Typst templates; model import runs only during export. This additional publication asset uses the existing GammaLoop package and does not change the documentation Cargo producer.

Manual Pages dispatches with `publish=false` build and seed this cache without merging Pages history or configuring, uploading, or deploying GitHub Pages. A docs-only successor run can therefore validate cross-run reuse without changing the published site.

The first optimized cold seed, run `32041014452` at `4f955f5a029b`, built the single Cargo producer in 15 minutes 16 seconds and the terminal Pages derivation in 2 minutes 58 seconds. The complete site step took 18 minutes 53 seconds, down from 71 minutes 2 seconds in the three-layer cold seed `32028483780`. Hestia then uploaded the ten-path, 1.2 GiB root in 12.1 seconds.

The docs-only successor, run `32042462853` at `9d305a0154c0`, kept that producer's derivation and output paths unchanged. Its build plan contained only the new Pages derivation and restored the producer from Hestia. The site step took 4 minutes 10 seconds, including a 3-minute-4-second Pages build, and the complete job took 6 minutes 33 seconds. The drain uploaded only the new 1.3 MiB Pages path.

Follow-up: cross-job producer barriers

The cold NixCI fan-out exposed a scheduling problem outside Cargo and Crane: identical hidden derivations could start on separate workers before either copy reached the shared cache. The manual graph is still defined over all flake outputs, including `doNotBuild` outputs, but its NixCI projection now contracts each hidden path to the nearest job that NixCI actually builds. It stops at that first producer rather than adding the entire transitive closure as direct edges.

An explicit producer attribute alone is not a sufficient barrier. NixCI memoizes a successful top-level derivation across commits and can report `Nothing to build: this was already built successfully before.` without asking a worker to realize its output or closure. If that nested Cargo artifact is no longer available to the next workers, every released consumer can still build the same derivation. A replacement run exposed 15 exact derivations built in two archive jobs each. Five were test-dependency producers and accounted for 25 redundant `Compiling` lines.

Scheduled Cargo artifact outputs are therefore exposed through a tiny revision-scoped barrier derivation. The barrier output is a symlink to the stable raw artifact, so building it realizes the artifact and lets NixCI's post-build hook publish the closure before dependent jobs start. Only the wrapper contains the flake revision: the Cargo artifact derivation and its cross-commit cache key remain unchanged. This adds no target-tree copy and preserves paths such as `${artifact}/target.tar.zst` through the output symlink. The wrappers cover the global artifact, package dependency artifacts, ordinary and contextual test dependencies and binaries, the Python module artifact chain, and the other scheduled shared Cargo artifacts. Hidden aliases such as the unused `spynso3` test outputs remain hidden.

The ordinary Rust API and Python ABI lanes share `crate-deps-gammalooprs`, so that artifact is now an explicit producer barrier. Both lanes wait for this sequence before their distinct Cargo contexts branch:

```
crate-deps-gammaloop-workspace-hack
  -> cargoArtifacts
  -> crate-deps-gammalooprs
  -> {gammaloopApiPackageArtifacts, gammaloop-python-module}
```

Every non-hack `crate-deps-*` node also records its real `cargoArtifacts` dependency in the scheduling graph. The duplicate final-package and test-binary aliases for the workspace hack are hidden, leaving its `crate-deps-*` attribute as the single producer for that exact derivation. The `checks.gammaloop` alias is also hidden because it is the same derivation as `packages.gammaloop`.

The nextest scheduling graph now matches the flake contexts:

- the Spenso archive waits for `symbolica-utils`;
- the Python API archive waits for a stable, target-named output for the `python-api-tests` integration binary and its distinct dependency context;

- the unconsumed `spynso3` test-dependency and test-binary outputs are not NixCI jobs.

Grouped archive commands were the remaining Cargo-context mismatch. Package test binaries are deliberately built from package-local sources and features, so each package now has a nextest archive derivation with the same singleton Cargo arguments. A lightweight link farm retains the existing group output. The corresponding group test job executes those archives sequentially, reports each JUnit result, and returns the first failing status. This retains one NixCI archive and test job per existing group without asking Cargo to reunify all packages in a new grouped compilation context or letting one archive command mutate another package's Cargo context.

Three redundant materialization layers were also removed. Final crate builds consume their self-contained, stripped `crate-deps-*` archive directly, and the Python build and package phases consume their preceding self-contained archive instead of routing it through a one-input `mergeCargoArtifacts`. The terminal nextest archives consume the matching package-local incremental artifact and deep-copy inherited artifacts before overlaying the package-specific delta. Cargo can decide that a transitive workspace unit needs refreshing when documentation or another source-only input changes; a store-backed symlink would make the resulting `.rmeta` write fail. Its `target.tar.zst.prev` link now targets an archive file, so Crane's normal post-patch hook restores the entire chain. This avoids both a new materialized group merge and publication of another Cargo target tree while keeping the terminal target writable.

The self-contained archive compaction step uses mtime epoch 1, matching Crane's artifact installer and Nix source timestamps. Using epoch 0 for the compacted target made Cargo treat every inherited artifact as older than its source when a consumer skipped the materializing merge layer.

Nextest archives must also contain everything their tests need after the build directory has disappeared. Generated shell and Python catalogue fixtures are therefore embedded in the docs-example test binary with `include_str!` rather than opened through `OUT_DIR` at runtime. Tests such as `trybuild` deliberately launch Cargo again; the archive runner gives those nested invocations an offline `CARGO_HOME` containing the same vendored Cargo configuration used to compile the archive. The docs-builder tests additionally consume the authored `docs/` tree, linked Python notebook sources, transitive Typst and graph inputs, checked-in changelogs, and the verified `flake.nix` scope at runtime, so the docs group carries those files in its remapped source. Latest and snapshot rendering also receive linked notebook sources so source-link validation sees the same repository inputs. Clinnet's embedded Typst test fixtures belong to the compile-time source set; those inputs remain present in the remapped runtime source. Its runner also provides the pinned Typst CLI, Git for provenance hashes, and Python for syntax-checking generated examples. The filtered test source has no `.git` directory, so the runner supplies stable, explicitly non-publishing commit and timestamp fixtures for generated-page tests. It also gives tempfile-based containment tests a dedicated `TMPDIR` below `target/` rather than exposing Nix's broad `/build` root as the persistent-cache namespace.

The latest Pages tree and the immutable snapshot fixture are now separate, independently cacheable terminal derivations over the same Cargo producer. The lightweight full documentation check consumes both completed trees, installs them through the Pages history updater, and asserts the latest and snapshot routes for all six products together with the portal, developer pages, and generated API material. The manual NixCI graph orders both render jobs after the shared documentation Cargo artifact and orders the merge check after both renders, preventing concurrent consumers from rebuilding the same producer.

This separation is required by the hosted worker boundary rather than by a source failure. The former monolithic check still exited after 13 minutes 14 seconds immediately after completing its

second all-product render, even after a third redundant render had been removed. Keeping each render in its own derivation lets NixCI cache and schedule it independently, while the merge policy remains covered without rerendering either tree. Lower-level renderer tests retain deterministic-output and persistent-session coverage.

The merge derivation provides Bash explicitly because the updater is a child shell process. Its first test intentionally captures an expected failure; if `bash` is absent from `PATH`, that command-not-found error is captured instead and the following message assertion fails without useful output. Shell tracing keeps future route-policy failures attributable to their exact assertion. Its cross-project link assertion uses the relative URL emitted by a full-site build; only isolated product previews rewrite that URL to the absolute public origin.

The static graph and derivation shapes can be checked without building the cold Cargo closure:

```
nix eval --json --file nix-ci.nix
nix flake check --impure --no-build 'path:.'
nix derivation show \
  'path:.#checks.x86_64-linux.gammaloop-nextest-binaries-spenso'
```

A fresh NixCI run is still required to measure cross-worker execution counts and cache upload time; one local Nix daemon serializes identical derivations and therefore cannot reproduce the original stampede.

Follow-up: toolchain refresh source and target boundaries

The primary Nixpkgs and Rust inputs can advance without changing the documentation renderer: `nixpkgs-docs` retains the reviewed Typst 0.15 toolchain and fonts, while the main package set supplies the development shell and Rust CI toolchain. This split lets `nix develop` follow the current package set without silently changing rendered Pages output.

The documentation catalog's production source slice is derived from the workspace-relative `source = "..."` annotations in `annotated_items.rs`. Those paths are compile-time inputs to the catalog proc macros, not normal Rust dependencies. Deriving the exact list preserves narrow invalidation while preventing a newly annotated API item from compiling in the full workspace but disappearing from the isolated `crate-deps-*` build.

Embedded test graphs belong to the compiling test package's source slice. In particular, the GammaLoop core slice includes the GL256 and GL262 three-loop four-photon fixtures used with `include_str!`. Its RQFT renormalization graphs are runtime inputs instead. Declaring these files at their owning boundaries keeps isolated Nix test runs equivalent to a complete checkout without widening unrelated production sources. The API and integration packages also declare the frozen GL15 graph as a runtime input: their metadata and event checks import the same physical pentagon independently of generator numbering.

The Python ABI artifact family now selects Cargo library targets explicitly. Its `pyo3-extension-module` feature intentionally leaves Python symbols for the interpreter to resolve when the shared object is loaded. Building the `gammaloop` executable in that same feature context is therefore invalid; `--lib` excludes the CLI without changing the extension's dependency or feature identity. The ordinary CLI remains in the separate Rust package artifact family, where it is linked with its normal feature context.

Follow-up: Community packages and explicit non-Rust inputs

The documentation source set carries the Community graph stub, the `hepkit` wrapper and stub, and the host graph and tensor mirrors. It includes the full Linnet Python test directory, matching discovery in the terminal consumer. The explicit documentation input list drops `crates/linnet-py/`

`pyproject.toml` and its `uv.lock`; Cargo manifests and shared stubs remain inputs through their owning source sets. Both the dependency manifest source set and the documentation Cargo source set include `localCargoPatchSources`, so a declared local Cargo patch is available in an isolated build and changes its own dependency input identity. Selection follows root Cargo path-patch declarations; an undeclared vendor directory does not enter source or producer identity.

The package source map retains separate production, compile-time test, and runtime owners. Production inputs now include interactive SVG and collection CSS/JavaScript, UFO parameter-label code, the Linnet Typst tree, layout Wasm plugins with their license files, and renderer fonts. Embedded Python rendering tests and graph/layout fixtures enter their own compile-time test slice, including the Community graph stub used by the exporter regression. Nauty driver and answer files enter Linnet's runtime slice. These declarations follow the actual include/read owner; they do not turn a crate's test fixtures into unrelated production inputs. SOURCE30 explicitly includes the request-decorator Python contract, five Linnet generation-report snapshots and four Clinnet literal Typst fixtures. Current combined producers and input sensitivity were independently approved at `7587d0c7`; that focused closure proof does not imply full NixCI or runtime-physics coverage.

Remaining caveats

- Dependency-only derivations can still compile generated feature-anchor or consumer crates and root-specific third-party crates. The public `crate-deps-*` package outputs now preserve the real current package artifact, and `crate-deps-gammaloooprs` does not compile `linnet`, `spenso`, `idenso`, `vakint`, `spenso-hep-lib`, or `gammaloop-tracing-filter`.
- Generated test dependency anchors are build-source-only workspace members. Because they are not in `Cargo.lock`, support, test-binary, and nextest archive commands use `--offline` rather than `--locked`.
- Workspace manifests and `Cargo.lock` remain coarse inputs to every source tier. Inline `#[cfg(test)]` blocks also remain part of production `.rs` source hashes; moving them to separate modules is the way to narrow that boundary.
- Python ABI builds intentionally use different `gammaloop-api` features (`python_abi`, `pyo3-extension-module`) and remain a separate artifact family. Normal Rust test archives should not depend on that family.
- `doNotLinkInheritedArtifacts = true` remains on producer paths whose archive and stripping behavior still needs a focused chain test before switching to Crane's symlink-heavy inheritance mode. Terminal nextest archives also retain it because Cargo may refresh transitive workspace metadata there. Deep copies should not cause recompilation, but they can add wall time. The next publication experiment should compare Crane's `use-symlink` artifact-install mode across earlier root-to-package-to-test stages without reintroducing read-only terminal artifacts.